

Preliminary Requirement Analysis

SE 26S Team 4

1 Introduction

We choose to implement a **Student Productivity Agent** as our software development project.

2 Functional requirements

Our proposed **Student Productivity Agent** will feature 5 distinct, high-level functionalities driven by an agentic loop:

- **Multi-Source Integration:** The agent actively monitors and consolidates data from diverse channels, including Blackboard deadlines, personal TODO lists, and the user's university mailbox. It extracts deadlines and announcements from emails to autonomously update schedules and resolve conflicts, and can even draft contextual replies.
- **Campus Knowledge Retrieval (Campus Encyclopedia):** A RAG-based tool that digests official university handbooks, such as the SUSTech University Calendar. It can answer complex student queries (e.g., "What is the deadline for changing my dorm?" or credit requirements) by searching through official documents.
- **Study Material Copilot:** A document processing tool where the agent reads uploaded lecture PPTs or Markdown notes and automatically generates review summaries or practice quizzes based strictly on the uploaded materials.
- **System-Level Task Automator:** The agent will have access to local file system tools to perform OS-level tasks via natural language, such as batch-renaming lab submission files to the required `[Name_ID_Lab1].zip` format.
- **Agentic GUI with Human-in-the-Loop:** A custom graphical interface that includes three main components: a chat/visualization area for interaction, a dedicated "Thought Trace" panel to show the LLM's reasoning and tool-selection process, and an explicit confirmation mechanism (Human-in-the-loop) that blocks the agent from executing risky actions (like modifying schedules or renaming files) until the user approves.
- **Context-Aware Student Profiling:** Instead of a static information dashboard, the system builds a dynamic personal profile capturing the user's academic status (e.g., major, GPA) and lifestyle preferences. The agent securely leverages this profile as background context during its reasoning phase, ensuring that schedule optimizations, campus knowledge retrieval, and study plans are highly personalized without requiring the user to repetitively provide their basic background in the chat.

3 Non-functional requirements

- **Safety & Security:** The system must strictly enforce the human-in-the-loop policy. It cannot have autonomous write-access to the local system or external calendars without triggering a user permission prompt.
- **Usability:** The GUI must clearly separate the technical "Thought Trace" logs from the main chat interface so non-technical users aren't overwhelmed by JSON outputs or backend logic.
- **Performance:** Tool execution should be relatively fast. Simple handbook queries should return in under 3 seconds, while heavy document processing (like parsing a 50-page PPT) should provide a clear progress indicator.

4 Technical requirements

4.1 Tech Stack

- **Backend Stack:** We plan to build the core API using FastAPI and Python. For data storage, we are considering PostgreSQL. To implement the agent's logic and tool integration, we will likely use llama index as our main framework.
- **Frontend Stack:** For the user interface, our current idea is to use Vue 3 combined with Tailwind CSS for styling. We hope to write the frontend primarily in TypeScript to keep the code easier to maintain and avoid simple type errors.

4.2 Operating Environment

- **Development Environment:** We plan to use **VS Code** as our primary IDE on Windows/macOS. We will manage our code using **Git** and follow a layered architecture to ensure the project remains organized and maintainable.
- **Execution Environment:**
 - **Backend:** The server-side logic is expected to run in a **Python 3.10+** environment. It requires stable network access to reach LLM APIs (e.g., OpenAI) and the university's data sources.
 - **Frontend:** The user interface will be accessible via **modern web browsers** (such as Chrome, Edge, or Safari).
 - **Database:** We intend to host a local or cloud-based **PostgreSQL** instance for data persistence.

5 Data requirements

- **Data Needed:** SUSTech academic calendar events, Blackboard assignment data, official university handbooks (PDF/HTML format), user lecture slides (PPTX), and local file directories.

– **Data Acquisition:**

- Calendar data will be fetched via iCal URL feeds.
- Blackboard data will be acquired via web scraping or available API tokens.
- Handbooks and lecture slides will be provided by the user manually or placed in a specific local directory that the agent monitors and parses using Python libraries like PyMuPDF or python-pptx.